

Releasing a Signed and Notarized macOS App on GitHub Releases

Checked against Apple and GitHub documentation on August 14, 2026.

Goal

Publish **ToolSizeWatcher.app** outside the Mac App Store, through the GitHub **Releases** tab, in a way that passes macOS Gatekeeper.

For this project, the practical public artifact is:

- a signed, notarized, and stapled **.app**
- packaged inside a **.zip**
- uploaded manually as a GitHub Release asset

Do **not** upload the raw **.app** bundle directly to GitHub Releases.

What Gatekeeper Expects

For direct distribution outside the Mac App Store, Apple expects:

1. The app to be signed with a **Developer ID Application** certificate.
2. The app to use the **Hardened Runtime**.
3. The app to be **notarized** by Apple.
4. Ideally, the notarization ticket to be **stapled** to the app before distribution.

For this repository, that means:

1. Build the app.
2. Package the **.app**.
3. Sign the **.app**.
4. Submit a ZIP archive to Apple notarization.
5. Staple the ticket back onto the **.app**.
6. Create a fresh final ZIP from the stapled **.app**.
7. Upload that final ZIP to GitHub Releases.

Important Membership Note

You said you already have an Apple Developer account and only need to renew the yearly payment.

That renewal matters before you start shipping signed updates.

Apple states that apps already signed with a valid Developer ID certificate can continue to run even after membership expiration, but you need an active Apple Developer Program membership to obtain new Developer ID certificates for future builds and updates. For a normal release workflow, assume your membership must be active.

Recommended Release Format

For GitHub Releases, the simplest format is:

- `ToolSizeWatcher-macos-arm64-v0.1.0.zip`

Or, if you later produce a universal binary:

- `ToolSizeWatcher-macos-universal-v0.1.0.zip`

Also recommended:

- `ToolSizeWatcher-macos-arm64-v0.1.0.zip.sha256`

Why ZIP:

- GitHub Releases accepts uploaded binary assets directly.
- A ZIP preserves the `.app` bundle structure when created with `ditto`.
- Apple notarization accepts ZIP archives.

Prerequisites

Apple Side

- Active Apple Developer Program membership
- A **Developer ID Application** certificate installed in your macOS keychain
- Your Apple Developer Team ID
- Xcode or Xcode Command Line Tools installed

This project does **not** currently need:

- Mac App Store distribution
- App Store Connect app records
- a `Developer ID Installer` certificate
- a provisioning profile, unless you later add advanced capabilities that require one

GitHub Side

- Push access to the repository
- Permission to create releases in the repository
- Each uploaded release asset must stay under GitHub's per-file limit of 2 GiB

One-Time Setup

1. Renew Apple Developer Membership

Complete the yearly Apple Developer Program renewal first.

Without that, you risk being blocked when:

- creating or renewing Developer ID certificates
- notarizing new builds

- shipping future updates

2. Create or Confirm the Developer ID Certificate

In Apple Developer account management, create or verify a:

- **Developer ID Application**

Install the certificate into your login keychain.

Then verify that macOS sees it:

```
security find-identity -v -p codesigning
```

You should see a usable identity similar to:

```
Developer ID Application: Your Name or Company (TEAMID1234)
```

3. Create Notary Credentials in the Keychain

For a manual workflow, the simplest option is usually:

- Apple ID
- app-specific password
- Team ID
- stored locally with **notarytool store-credentials**

Create an app-specific password for your Apple ID, then store the profile:

```
xcrun notarytool store-credentials "tool-size-watcher-notary" \  
  --apple-id "you@example.com" \  
  --team-id "TEAMID1234" \  
  --password "app-specific-password"
```

Test that the profile works:

```
xcrun notarytool history --keychain-profile "tool-size-watcher-notary"
```

If you prefer, you can later switch to an App Store Connect API key workflow. For a manual release process, the keychain profile is usually enough.

Release Workflow

1. Build the App

Rebuild the binary:

```
./rebuild.sh
```

Create the unsigned **.app** bundle:

```
./scripts/package_macos_app.sh
```

At this point, the bundle should exist at:

```
dist/ToolSizeWatcher.app
```

Important:

- the current packaging script creates the **.app**
- it does **not** sign it
- signing and notarization happen after packaging

2. Set Release Variables

Example:

```
VERSION="0.1.0"
ARCH="arm64"
APP="dist/ToolSizeWatcher.app"
IDENTITY="Developer ID Application: Your Name or Company (TEAMID1234)"
NOTARY_PROFILE="tool-size-watcher-notary"
UPLOAD_ZIP="dist/ToolSizeWatcher-for-notarization.zip"
RELEASE_ZIP="dist/ToolSizeWatcher-macos-${ARCH}-v${VERSION}.zip"
```

If you later build a universal binary, replace **ARCH="arm64"** with **ARCH="universal"**.

3. Sign the **.app**

For the current project, a top-level app signing step is enough because the app does not currently bundle nested frameworks or plug-ins.

Sign it with:

```
codesign --force \
  --options runtime \
  --timestamp \
```

```
--sign "$IDENTITY" \  
"$APP"
```

Verify the signature:

```
codesign --verify --deep --strict --verbose=2 "$APP"  
codesign -dv --verbose=4 "$APP" 2>&1
```

Things to look for:

- the correct **Developer ID Application** identity
- a secure timestamp
- Hardened Runtime enabled

If you later add embedded frameworks, helper apps, or plug-ins, sign those nested components first, then sign the outer **.app**.

4. Create the Notarization Upload Archive

Use **ditto**, not Finder compression, so the app bundle metadata is preserved correctly:

```
rm -f "$UPLOAD_ZIP"  
ditto -c -k --sequesterRsrc --keepParent "$APP" "$UPLOAD_ZIP"
```

This ZIP is for Apple notarization submission.

Do **not** assume it is the final release asset yet.

5. Submit to Apple Notarization

Submit and wait:

```
xcrun notarytool submit "$UPLOAD_ZIP" \  
--keychain-profile "$NOTARY_PROFILE" \  
--wait
```

If Apple accepts the submission, continue.

If Apple rejects it, inspect the log:

```
xcrun notarytool log <submission-id> \  
--keychain-profile "$NOTARY_PROFILE"
```

Typical problems:

- unsigned nested code
- missing hardened runtime
- invalid entitlements
- post-signing modification of the app bundle

6. Staple the Notarization Ticket

Once notarization is accepted, staple the ticket onto the `.app`:

```
xcrun stapler staple "$APP"
xcrun stapler validate "$APP"
```

This is important because:

- Gatekeeper can validate more reliably offline
- users get a better first-run experience
- your public release asset should contain the stapled app, not just the notarized submission archive

7. Run Local Gatekeeper Checks

Run:

```
spctl --assess --type execute --verbose=4 "$APP"
codesign --verify --deep --strict --verbose=2 "$APP"
```

Expected result:

- `accepted` from `spctl`
- a valid signature from `codesign`

8. Create the Final Public ZIP

After stapling, build a **new** ZIP for distribution:

```
rm -f "$RELEASE_ZIP"
ditto -c -k --sequesterRsrc --keepParent "$APP" "$RELEASE_ZIP"
shasum -a 256 "$RELEASE_ZIP" > "${RELEASE_ZIP}.sha256"
```

This step matters.

Do **not** upload the pre-stapling notarization ZIP as your public asset.

The release asset should be produced **after** stapling.

9. Publish Through the GitHub Releases Tab

Use the GitHub web UI:

1. Push the commit or tag you want to release.
2. Open the repository on GitHub.
3. Open the **Releases** page.
4. Click **Draft a new release**.
5. Choose or create the tag, for example `v0.1.0`.
6. Set a release title, for example `Tool Size Watcher v0.1.0`.
7. In the asset area, upload:
 - `ToolSizeWatcher-macos-arm64-v0.1.0.zip`
 - `ToolSizeWatcher-macos-arm64-v0.1.0.zip.sha256`
8. Add release notes.
9. Publish the release.

Recommended notes to include:

- supported macOS version
- supported architecture: `arm64` or `universal`
- whether the release is signed and notarized
- checksum instructions

If your repository uses immutable releases, create the release as a draft first, upload all assets, then publish.

10. What Users Should Download

Users should download the uploaded release asset:

- `ToolSizeWatcher-macos-arm64-v0.1.0.zip`

They should **not** rely on:

- GitHub's automatic source ZIP
- GitHub's automatic source tarball

Those source archives are repository snapshots, not signed app distributions.

11. Post-Release Validation

Before announcing the release, test the exact downloaded asset from GitHub:

1. Download the ZIP from the published release page.
2. Unzip it on a separate Mac or a clean macOS user account.
3. Confirm the app opens normally from Finder.
4. Confirm Gatekeeper does not block it.

Best practice:

- test on a machine that has never seen the app before
- test the actual file downloaded from GitHub Releases

That catches issues such as:

- wrong asset uploaded
- asset renamed incorrectly
- stapling done after the uploaded ZIP was created

Common Mistakes

Uploading the Wrong ZIP

Wrong:

- sign app
- zip app
- notarize ZIP
- staple app
- upload the old ZIP

Correct:

- sign app
- zip app
- notarize ZIP
- staple app
- create a new ZIP
- upload the new ZIP

Using the Wrong Certificate

For a zipped **.app**, use:

- **Developer ID Application**

Not:

- **Developer ID Installer**

The installer certificate is for signed **.pkg** installers.

Forgetting Hardened Runtime

For Developer ID notarization, Hardened Runtime must be enabled.

In a CLI signing flow, that is the purpose of:

```
--options runtime
```

Modifying the App After Signing

Anything that changes the **.app** contents after signing can invalidate the signature.

Do not:

- edit files inside the bundle
- add resources after signing
- repackage the app contents manually

If anything changes, sign again and redo notarization.

Assuming Membership Expiration Is Harmless

It is true that already-signed apps can continue to run if they were signed while the certificate was valid.

It is **not** safe to plan releases around an expired membership, because future signing, certificate renewal, and update distribution can be blocked.

Suggested Manual Checklist

Before each public release:

- Apple Developer membership is active
- **Developer ID Application** certificate exists in Keychain
- **./rebuild.sh** succeeded
- **./scripts/package_macos_app.sh** succeeded
- **.app** signed successfully
- notarization accepted
- ticket stapled successfully
- **spctl --assess** says **accepted**
- final post-stapling ZIP created
- SHA-256 file created
- final ZIP uploaded to GitHub Release
- downloaded GitHub asset tested on a clean machine or clean user account

Recommended Future Improvement

Later, you can automate most of this with:

- a dedicated release shell script
- or a GitHub Actions workflow running on macOS

But for now, a manual GitHub Release flow is perfectly valid, and easier to debug while the app is still evolving.

References

- Apple: Signing your apps for Gatekeeper
<https://developer.apple.com/developer-id/>
- Apple: Notarizing macOS software before distribution
<https://developer.apple.com/documentation/security/notarizing-macos-software-before-distribution>
- Apple: Developer ID certificates
<https://developer.apple.com/help/account/certificates/create-developer-id-certificates/>

- Apple: Hardened Runtime
<https://developer.apple.com/documentation/security/hardened-runtime>
- Apple: Creating distribution-signed code for macOS
<https://developer.apple.com/documentation/xcode/creating-distribution-signed-code-for-the-mac>
- Apple: Customizing the notarization workflow
<https://developer.apple.com/documentation/security/customizing-the-notarization-workflow>
- Apple: Resolving common notarization issues
<https://developer.apple.com/documentation/security/resolving-common-notarization-issues>
- GitHub: Managing releases in a repository
<https://docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository>
- GitHub: About releases
<https://docs.github.com/en/repositories/releasing-projects-on-github/about-releases>
- GitHub: Linking to releases
<https://docs.github.com/en/repositories/releasing-projects-on-github/linking-to-releases>